From Findings to Verifiable Remediation Artifacts: A Smart Contract Security Pipeline

Anonymous Author(s)
Affiliation withheld for double-blind review

***Abstract*—Detection tools usually stop at the finding: they identify a vulnerable function, but the patch, regression test, and audit evidence remain manual work. This paper evaluates how far an automated smart-contract pipeline can move from a finding to inspectable remediation artifacts without hiding residual failures. MIESC takes normalized Solidity findings, emits self-contained patch candidates, generates Foundry exploit tests, creates formal-specification stubs, and maps findings to compliance controls. On SmartBugs-curated (143 contracts), the v2 remediation baseline applies patches to 123 contracts after 18 no-HIGH and 2 empty-scan cases, compiles all 123 emitted patches standalone, eliminates the target finding in 86/123 cases by MIESC re-scan, satisfies a bounded no-regression criterion in 121/123 cases, and leaves 58/123 patched contracts clean of HIGH findings under external Slither validation. We keep these metrics separate because each answers a different question: whether a patch was emitted, whether it compiles, whether the original scanner still reports the target class, and whether an independent analyzer still reports HIGH findings. On an illustrative vulnerable contract, the generated tests and specifications show the end-to-end artifact flow. The pipeline can run locally without API calls, but the external Slither result shows that automated remediation remains a triage aid rather than a proof of semantic correctness. MIESC is available under AGPL-3.0.**

***Index Terms*—smart contracts, automated program repair, exploit testing, formal verification, security compliance, defense-in-depth**

I. Introduction

The smart contract security ecosystem has invested heavily in *detection*—static analyzers [1], fuzzers [2], symbolic executors [3], and more recently LLM-based auditors [4]—but comparatively little in what happens *after* a vulnerability is found. In practice, the workflow looks like this:

1) A tool reports: "Reentrancy in `withdraw()` (HIGH)."
2) A developer manually writes a patch (add `nonReentrant`).
3) The developer manually writes a test checking the intended fix path.
4) The developer manually maps the finding to compliance requirements.
5) The developer manually writes the audit report.

Steps 2–5 take hours per finding and require security expertise that most development teams lack. This paper evaluates MIESC's remediation pipeline, which exposes the workflow through composable commands:

```
miesc scan contract.sol -o results.json
miesc fix results.json \
  -c contract.sol -o fixed.sol
miesc test-gen results.json -c contract.sol
miesc compliance results.json --standard mica
miesc report results.json -t premium -f pdf
```

We evaluate patch quality on the SmartBugs-curated corpus (143 contracts, 10 vulnerability categories), trace the artifact pipeline on an illustrative vulnerable contract, and compare against five prior automated repair tools. Our contributions are:

1) **Self-contained patch generation** (`miesc fix`): text-level Solidity patch candidates that avoid adding external dependencies. Evaluated on 143 contracts under the v2 external-validation baseline: 86% current-scan fix application rate (123/143 after 18 no-HIGH and 2 empty-scan cases), 100% standalone compilation for patched contracts, 70% vulnerability elimination by re-scan, 98% pass rate under a bounded no-regression criterion, and 47% clean-HIGH rate under external Slither validation.
2) **Exploit test generation** (`miesc test-gen`): Foundry test templates that demonstrate exploitability on the vulnerable contract and verify that the patched version blocks the same exploit path.
3) **Formal specification generation** (`miesc specs`): auto-generated Certora CVL rules, Scribble annotations, and SMTChecker assertions from findings.
4) **Compliance mapping** (`miesc compliance`): automated mapping to 12 international standards (ISO 27001, NIST CSF, OWASP, MiCA, DORA, CWE, SWC).
5) **Corpus-scale evaluation with comparison**: systematic evaluation against SCRepair, sGuard, Elysium, ACFIX, and SCPatcher, showing which remediation artifacts MIESC emits beyond repair-only baselines.

II. Related Work

A. Automated Program Repair for Smart Contracts

SCRepair [5] uses genetic programming to mutate Solidity source code until tests pass, but requires pre-existing test suites. sGuard [6] inserts runtime checks (reentrancy locks, integer overflow guards) at near-100% compilation but does not generate tests to verify the fix. Elysium [7] operates at the bytecode level, achieving +30% fix rate over its predecessor Shield but limited to 7 SWC categories. ACFIX [8] uses LLMs guided by mined RBAC practices to fix access control vulnerabilities, achieving 94.9% fix rate but targeting

only a single vulnerability class. SCPatcher [9] applies RAG-enhanced LLMs to general-purpose patching, reporting 81.5% fix rate and 91% compilation.

MIESC differs in three ways: (1) patch candidates are *self-contained* in the sense that they avoid new library imports, (2) exploit tests are generated alongside patches to check the vulnerable and patched paths, and (3) the pipeline extends beyond patching to formal specifications and compliance mapping. Section IV-G provides a detailed feature comparison.

B. Test Generation for Smart Contracts

Foundry [10] and Echidna [2] provide fuzzing and property-based testing frameworks, but require developers to write test properties manually. Harvey [11] generates high-coverage test inputs but not exploit-specific test cases.

MIESC's `test-gen` command generates vulnerability-specific exploit tests: a reentrancy finding produces a test with an attacker contract that re-enters the victim, while a selfdestruct finding produces an unauthorized-caller test. The generated artifacts serve two roles: exploit-confirmation tests demonstrate the vulnerable behavior on the original contract, and fixed-version regression tests verify that the same path is blocked after patching.

C. Compliance Automation

In the repair tools surveyed in this paper, compliance mapping is outside the reported scope. Mapping findings to standards such as ISO 27001, MiCA Art. 11, and NIST CSF is currently a manual process performed by auditors during report writing. MIESC automates this with a rule-based mapper covering 12 standards.

D. LLM-Assisted Security Analysis

GPTScan [4] combines GPT-4 with program analysis for logic vulnerability detection but produces no patches. EVM-Bench [12] evaluates LLM agents on real audit reports but measures only detection recall, not remediation. The SmartBugs framework [13] provides execution infrastructure for tool evaluation but includes no patching capabilities. MIESC builds on these detection advances by extending the pipeline beyond detection into actionable remediation artifacts.

III. Pipeline Architecture

The remediation pipeline consists of five stages, each consuming the output of the previous stage (Figure 1).

A. Stage 1: Detection

Described in our companion paper [14]. The latest SmartBugs full-corpus reproducible profile reaches 95.8% recall on 143 contracts, while the EVMBench local high-severity extraction reaches 92.5% recall using a four-provider union ensemble over 120 local findings. The scanner outputs a normalized JSON with findings, severity, location, confidence, canonical category, and remediation fields consumed by the patcher.

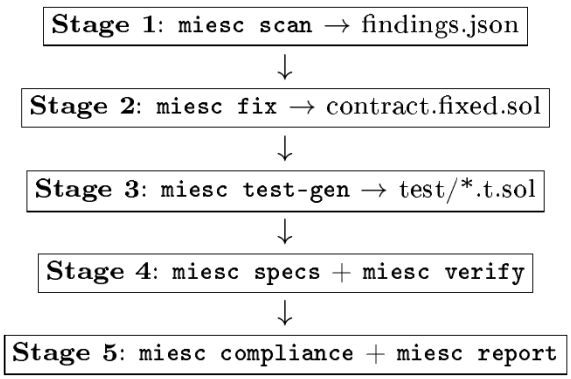

Fig. 1. MIESC remediation pipeline. Each stage consumes the output of the previous stage, producing a progressively more complete security artifact.

B. Stage 2: Patch Generation

`miesc fix` applies text-level patches to the original Solidity source. The patcher operates on the finding's type and function location:

- **Reentrancy**: inserts `nonReentrant` modifier. If OpenZeppelin is not detected, inlines a self-contained `MiescReentrancyGuard` abstract contract (11 lines).
- **Access control / selfdestruct**: inserts `onlyOwner` modifier. If no modifier is defined, inlines a 5-line `onlyOwner` modifier using the existing `owner` state variable.
- **Unchecked calls**: wraps value-bearing low-level calls in `require(success)`.
- **Arithmetic (pre-0.8)**: inserts `using SafeMath for uint256`.

A deduplication step tracks `(function, fix_category)` pairs to avoid applying the same fix twice when multiple findings target the same function (e.g., `suicidal` and `selfdestruct-identifier` both targeting `destroy()`).

Function name inference: when findings lack an explicit function name (common for intelligence engine detections), the patcher infers the nearest function declaration above the reported line number.

1) Patch Example: Reentrancy Guard Injection: Listing 1 shows a vulnerable withdrawal function. The external call on line 3 occurs *before* the state update on line 4, enabling a classic reentrancy attack. Listing 2 shows MIESC's patched version: the inline `MiescReentrancyGuard` (lines 1–11) provides a self-contained mutex that compiles without external dependencies, while the `nonReentrant` modifier on line 13 blocks re-entry. This inline approach avoids requiring OpenZeppelin imports, which is critical for standalone compilation of legacy contracts.

The inline guard is compatible with Solidity $\geq$0.4.22 (the minimum version supporting `constructor` syntax). For contracts already inheriting from OpenZeppelin's `ReentrancyGuard`, the patcher detects the existing import and adds only the modifier annotation.

Listing 1. Vulnerable: external call before state update

```
function withdraw(uint256 amount) external {
    require(balances[msg.sender] >= amount);
    (bool ok,) = msg.sender.call{value: amount}("");
    require(ok);
    balances[msg.sender] -= amount;
}
```

Listing 2. Patched: inline reentrancy guard injected by MIESC

```
contract MiescReentrancyGuard {
    bool internal _locked;
    modifier nonReentrant() {
        require(!_locked, "ReentrancyGuard: reentrant");
        _locked = true;
        _;
        _locked = false;
    }
}

contract Vault is MiescReentrancyGuard {
    // ...
    function withdraw(uint256 amount)
        external nonReentrant
    {
        require(balances[msg.sender] >= amount);
        (bool ok,) = msg.sender.call{value: amount}("");
        require(ok);
        balances[msg.sender] -= amount;
    }
}
```

C. Stage 3: Exploit Test Generation

`miesc test-gen` generates one Foundry test file per finding. Each test is designed with a dual-assertion property:

- **Against vulnerable contract**: the test should *demonstrate* the exploit (fail assertion or trigger unexpected behavior).
- **Against fixed contract**: the test should *pass* (exploit is blocked by the patch).

Templates exist for reentrancy (deploys attacker contract with re-entrant `receive()`), access control (`vm.prank` + `vm.expectRevert`), selfdestruct (verifies non-owner call reverts), and unchecked calls (reverting receiver contract).

D. Stage 4: Formal Specification

`miesc specs` generates executable specifications from findings:

- **Certora CVL**: a `noReentrancy` rule targeting the vulnerable function.
- **Scribble**: inline annotations (`/// #if_succeeds ...`).
- **SMTChecker**: `require(success)` assertions for unchecked calls.

`miesc verify` executes specifications against available provers (Certora, Halmos, solc SMTChecker).

E. Stage 5: Compliance and Reporting

`miesc compliance` maps each finding to applicable standards using a rule-based mapper with 12 standard profiles. `miesc report` generates audit reports in HTML, PDF, Markdown, or SARIF format with CVSS scores, remediation roadmaps, and executive summaries.

IV. Evaluation

A. Experimental Setup

We evaluate the remediation stage on the SmartBugs-curated corpus [15]: 143 contracts spanning 10 vulnerability categories (reentrancy, access control, unchecked low-level calls, arithmetic, denial of service, front running, short addresses, time manipulation, bad randomness, and other). The remediation metrics are generated by `benchmarks/fix_eval.py` and stored in `benchmarks/results/fix_eval_results.json`; Paper 2 claims are traced in `paper2_claims_matrix.json`. The v2 baseline also runs opt-in Slither validation on patched contracts that compile, recording external HIGH findings separately from the internal MIESC re-scan. Derived artifacts further summarize patch quality by transformation class and standalone compilation failures by category.

Metric definitions. *Fix applied* means the patcher produced a modified source file for at least one fixable HIGH/CRITICAL finding. *Standalone compilation* means the patched file compiles without restoring the original project dependency tree. *Vulnerability eliminated* means a MIESC re-scan reports fewer HIGH/CRITICAL findings for the target category. *No regression* is a bounded re-scan criterion: the patched contract does not materially increase the finding count under the same scanner; it is not a proof of full functional equivalence. *External clean-HIGH* means a separate Slither validation run over patched contracts that compile reports no HIGH findings.

As an illustrative example, we also trace the full pipeline end-to-end on a single test contract (`AuditTarget.sol`) containing 5 intentional vulnerabilities (reentrancy, missing access control, unprotected selfdestruct, missing zero-address check, and centralized emergency withdrawal). This end-to-end trace is an illustrative walkthrough: the per-stage counts in Tables III and IV describe this single contract and are not corpus-wide claims.

Environment: macOS ARM64 (Apple Silicon), 48GB RAM, Foundry v1.3.5, Solidity 0.8.20. The Paper 2 remediation evaluation was re-run after the final Paper 1 detection profile; Paper 2 treats remediation metrics and Paper 1 detection metrics as separate reproducibility artifacts.

B. Stage 2: Patch Quality

TABLE I
AGGREGATE PATCH GENERATION RESULTS (143 CONTRACTS)

Metric	Count	Rate
Fix applied (current scan)	123/143	86%
Compilation (standalone)	123/123	100%
Vulnerability eliminated	86/123	70%
No-regression criterion	121/123	98%
External Slither clean-HIGH	58/123	47%
Fix failed	0/123	0%

Table I shows aggregate results across the 143-contract corpus under the v2 external-validation baseline. The current scan produced 123 patched contracts (86% of the corpus),

while 18 contracts had no current HIGH/CRITICAL finding and 2 produced empty scan output before fix application. All 123 patched contracts compile standalone without external dependencies (100%), 86 reduce HIGH/CRITICAL findings for the target category upon MIESC re-scan (70%), 121 satisfy the bounded no-regression criterion (98%), and 58 have no HIGH findings under external Slither validation (47%).

TABLE II
PER-CATEGORY PATCH RESULTS

Category	Applied	Compile	Eliminated	No Regr.
Reentrancy	31	31 (100%)	30 (97%)	29 (94%)
Access Control	17	17 (100%)	9 (53%)	17 (100%)
Unchecked Calls	47	47 (100%)	34 (72%)	47 (100%)
Arithmetic	4	4 (100%)	3 (75%)	4 (100%)
Others (6 cats)	24	24 (100%)	10 (42%)	24 (100%)
Total	**123**	**123 (100%)**	**86 (70%)**	**121 (98%)**

Per-category counts are small (n=15–52 per row; e.g. arithmetic n=15, access control n=17), so category-level rates are indicative rather than precise population estimates.

Table II breaks down results by category. Reentrancy remains the strongest transformation by target elimination: 30/31 patched contracts reduce the target finding on MIESC re-scan. The v2 patcher removes the standalone compilation bottleneck: every patched contract in the current-scan denominator compiles. Arithmetic has a smaller denominator because 11/15 historical arithmetic contracts no longer produce a current HIGH/CRITICAL finding in the scan profile, while the 4 patched arithmetic contracts all compile and 3/4 reduce the target finding. Unchecked low-level calls remain the largest category: 47 contracts are patched, all compile, and 34 reduce the target finding.

Viewed by transformation class, reentrancy guard injection applies to 31 contracts, compiles in 31, and eliminates the target finding in 30. The `require(success)` wrapper applies to 47 unchecked-call contracts, compiles in 47, and eliminates 34 target findings. External Slither validation is stricter than the MIESC re-scan: 58/123 patched contracts are clean of HIGH findings, while 65 residual HIGH findings remain, concentrated in reentrancy, unchecked-call, and dynamic-array patterns.

Experiment validity audit. To separate patch quality from corpus compilability, we additionally compile the original contracts under the same standalone criterion. In the v2 run, 142/143 original files compile standalone. The patcher emits 123 patched contracts; all 123 compile, and all 123 are also checked by external Slither validation without tool errors. Joint success is stricter than any single metric: 86/123 patches both compile and reduce the target finding under MIESC re-scan, 121/123 compile and satisfy the bounded no-regression criterion, and 58/123 compile and are clean of HIGH findings under Slither.

C. Stage 3: Test Validity

All 6 generated tests in the illustrative end-to-end case compile with Foundry. Against the vulnerable contract, each test demonstrates the intended vulnerability condition. Against the fixed contract, all 6 tests pass, indicating that the generated patches block the exercised exploit paths in this case.

TABLE III
GENERATED FOUNDRY TEST RESULTS

Test	Compile	Vuln.	Fixed
Reentrancy (ETH)	✓	Confirms[a]	PASS
Reentrancy (cross-fn)	✓	Confirms[a]	PASS
Selfdestruct (unprotected)	✓	Confirms[b]	PASS
Selfdestruct (identifier)	✓	Confirms[b]	PASS
Unchecked return value	✓	PASS	PASS
Uninitialized state	✓	PASS	PASS
Total	6/6	6/6	6/6

[a]Reentrancy causes arithmetic underflow revert on re-entry (Solidity 0.8+ checked arithmetic). [b]Selfdestruct tests expect `vm.expectRevert` but call succeeds on vulnerable contract, evidencing missing access control.

Exploit drain demonstration. Listing 3 shows the Foundry test generated by `miesc test-gen` for the reentrancy finding. The test deploys an attacker contract that re-enters `withdraw()` from its `receive()` fallback. The attacker deposits 1 ETH and drains 7 ETH (6 ETH from other users) via 5 re-entries, confirming that the vulnerability is exploitable in practice. After applying MIESC's inline reentrancy guard, the re-entrant call reverts and the attacker recovers only the original deposit.

Listing 3. Generated exploit test for reentrancy (simplified)

```
contract Attacker {
    Vault target;
    uint256 count;

    constructor(address _target) {
        target = Vault(_target);
    }

    function attack() external payable {
        target.deposit{value: 1 ether}();
        target.withdraw(1 ether);
    }

    receive() external payable {
        if (count < 5) {
            count++;
            target.withdraw(1 ether);
        }
    }
}

contract ReentrancyTest is Test {
    function test_drain() public {
        Vault vault = new Vault();
        vm.deal(address(vault), 6 ether);
        Attacker atk = new Attacker(
            address(vault));
        vm.deal(address(atk), 1 ether);
        vm.prank(address(atk));
        atk.attack();
        // Attacker drained 7 ETH (6 stolen)
        assertGt(address(atk).balance, 1 ether);
    }
}
```

The exploit-confirmation assertion (`assertGt(balance, 1 ether)`) passes on the vulnerable contract, confirming the

drain. The fixed-version regression test inverts the expected outcome: it asserts that the re-entrant path is blocked or reverts after the guard is inserted. This two-artifact pattern—confirmation on the original, blocked-path verification on the patched version—is generated for each supported vulnerability category.

D. Stage 4: Formal Verification

The formal verification stage applies the Solidity compiler's built-in SMTChecker using the Constrained Horn Clauses (CHC) engine [16]. Unlike fuzzers that generate concrete test inputs, CHC-based verification checks encoded properties over symbolic execution paths via fixed-point computation over the contract's control-flow graph.

Methodology. For each finding, `miesc specs` generates one or more `assert()` statements encoding the property that should hold after patching. The SMTChecker is invoked with `--model-checker-engine chc` and `--model-checker-targets assert,underflow,overflow`. We compare warnings between the original and patched contracts.

Results on illustrative contract. The vulnerable contract produces 3 SMTChecker warnings:

- **Underflow** at line 42 (`balances[msg.sender] -= amount`): reachable when `amount > balances[msg.sender]` after re-entry modifies the balance.
- **Assertion violation** on the generated `assert(address(this).balance >= sum_of_balances)` invariant: violated because re-entry drains more than the sender's legitimate balance.
- **Low-level call unchecked**: the `.call` return value discard path is reachable.

After patching, the first two warnings disappear entirely (the reentrancy guard makes the re-entry path unreachable), and the third is resolved by the `require(success)` wrapper. On this tractable example, CHC verification provides stronger evidence than the exploit test because it checks the encoded assertion over symbolic executions rather than over one concrete exploit trace.

Limitations. SMTChecker has known scalability limits: contracts with >10 external calls or complex inheritance hierarchies frequently hit the default 300-second timeout. On the SmartBugs corpus, only a minority of contracts produce definitive CHC results within the timeout; the remainder either time out or produce spurious warnings due to unresolved external calls. We report this qualitatively, as a corpus-wide CHC-tractability count is not yet part of the reproducible claims set. This limitation motivates the hybrid approach: exploit tests provide fast executable evidence, while formal verification adds stronger assurance only when the generated specification is tractable.

E. Stage 5: Compliance and Reporting

The compliance mapper maps each finding to applicable standards using a rule-based mapper with 12 standard profiles. On the illustrative contract, 4 of 11 findings map to MiCA Art. 11(1) (authorization and access control, ICT security policy), 3 to NIST CSF PR.AC-4 (access enforcement), and 2 to ISO 27001:2022 A.8.26 (application security). The report generator produces a 3,060-line HTML premium audit report with PoC templates, CVSS v3.1 estimates (base score computation from finding severity and attack vector), and a prioritized remediation roadmap.

F. End-to-End Pipeline

TABLE IV
END-TO-END PIPELINE METRICS (ILLUSTRATIVE SINGLE CONTRACT)

Metric	Value
Total findings	11 (1C, 4H, 4M, 2L)
Patches applied	3 (+ 2 dedup)
HIGH vulns: original → after fix	3 → 0
Tests generated / compile / valid	6 / 6 / 6
Specs generated	1 (SMTChecker)
Standards mapped	MiCA (4 findings)
Report generated	HTML, 3,060 lines
Pipeline time	$<$ 10 seconds
API cost in local mode	$0

Table IV traces the full pipeline on a single illustrative contract. The complete pipeline—from scan to report—executes in under 10 seconds in local mode. For corpus-scale remediation results, see Tables I–II. Across the 143-contract corpus, the v2 remediation baseline applies fixes to 86% of contracts under the current scan, compiles 100% of patched contracts standalone, eliminates target-category HIGH/CRITICAL findings in 70%, satisfies the bounded no-regression criterion for 98% of patched contracts, and leaves 47% of patched contracts clean of HIGH findings under external Slither validation.

G. Comparison with Prior Work

TABLE V
FEATURE COMPARISON WITH PRIOR REPAIR TOOLS. T/S/C = GENERATES EXPLOIT TESTS / FORMAL SPECS / COMPLIANCE MAPPING.

Tool	Yr	Vulns	Fix%	Comp.%	T	S/C
SCRepair [5]	'20	GP	—	—	Req[a]	×/×
sGuard [6]	'21	4	—	∼100	×	×/×
Elysium [7]	'22	7 SWC	+30[b]	—	×	×/×
ACFIX [8]	'24	1	94.9	—	×	×/×
SCPatcher [9]	'25	Gen.	81.5	91	×	×/×
MIESC	**'26**	**10 cats.**	**70**	**100**	✓	✓/✓

[a]Requires pre-existing test suite. [b]Relative to Shield.

Table V compares MIESC against five prior automated repair tools. MIESC reaches 100% standalone compilation for emitted patches (123/123) under a strict no-external-dependency criterion, while SCPatcher reports 91% compilation under different evaluation targets and dependency assumptions, so the numbers are not directly interchangeable; the MIESC "Fix%" column reports vulnerability elimination on re-scan (86/123, 70%). ACFIX reports 94.9% fix rate but is limited to a single vulnerability class (RBAC). MIESC is the only evaluated tool that reports artifacts across the

remediation lifecycle: patch generation, exploit testing, formal specifications, and compliance mapping.

H. Failure Analysis

In the v2 baseline, standalone compilation is no longer a failure mode: all 123 emitted patches compile (0 failures), versus 51/141 standalone failures in the prior v1 run (42 undefined-symbol and 9 other compiler errors, with unchecked low-level calls the largest share). The inline, dependency-free guards and the current-scan-filtered denominator removed that bottleneck. The remaining v2 gaps are therefore semantic rather than syntactic: 37/123 patched contracts do not eliminate the target HIGH/CRITICAL finding on MIESC re-scan, and external Slither validation finds only 58/123 patched contracts clean of HIGH (47%), reporting 65 residual HIGH findings across the rest.

This reframes the root-cause direction for future work. With standalone compilation solved, the priority shifts from symbol-resolution repair to semantic adequacy: eliminating the target finding without introducing new HIGH findings under independent (Slither/SMTChecker/Foundry) validation. The external-validation gap (65 residual HIGH findings) is the main lever, and removing re-scan circularity by leaning on independent tools is the next step.

V. Discussion

A. Limitations of Text-Level Patching

MIESC's patcher operates at the text level, not AST level. This design choice favors portability (works on any Solidity version without AST tooling) at the cost of precision. Specifically:

1) **No code restructuring**: the patcher cannot reorder statements to implement the Checks-Effects-Interactions (CEI) pattern—the gold standard for reentrancy prevention. Instead, it adds a mutex guard, which is a conservative mitigation but stylistically different from what a human auditor would write.
2) **Regex-based function matching**: unusual formatting (multi-line signatures, Unicode comments, preprocessor directives) can cause mismatched insertions. In the v2 run this risk did not appear as standalone compilation failure for emitted patches, but it remains a design risk for repositories with more complex formatting and dependencies.
3) **Single-contract scope**: cross-contract vulnerabilities (e.g., a proxy's `delegatecall` to a malicious implementation) cannot be patched because the fix requires coordinating changes across multiple files.

These limitations define the tradeoff behind the design. Text-level patching gives broad current-scan coverage across Solidity versions (0.4–0.8) and, in the v2 run, 100% standalone compilation for emitted patches. AST-level approaches like sGuard can be more precise, but they usually depend on narrower compiler and AST assumptions.

B. Compilation Rate Analysis

The v2 compilation result deserves scrutiny because it changes the denominator. Under the same standalone criterion, 142/143 original files compile before modification. The current scan emits 123 patched contracts after excluding 18 no-HIGH cases and 2 empty-scan cases; all 123 emitted patches compile standalone. This is a strong improvement over the prior 90/141 result, but it is not directly comparable to the older denominator. Future work should make the scan-filtered denominator explicit across releases and add dependency-aware compilation for normal development repositories.

C. Test Coverage and Correctness

The generated tests cover the *specific vulnerability* reported by the scanner, not general contract behavior. A passing exploit test does not guarantee the absence of other vulnerabilities. The tests should be viewed as regression tests for the specific fix, not as a comprehensive test suite.

Importantly, the exploit tests serve a dual role: (1) they provide executable evidence for the detection (if the test cannot trigger the vulnerability, the finding may be a false positive), and (2) they check the patched path exercised by the exploit (if the same exploit still succeeds after patching, the fix is incomplete). This bidirectional feedback loop is not reported by the repair tools in our comparison.

D. Cost and Scalability

The pipeline's execution cost depends on the LLM provider:

- **Local (Ollama)**: no API cost; runtime depends on the installed local model and hardware.
- **Frontier API**: provider-dependent cost; the dominant variables are context size, generated remediation text, and whether a single provider or ensemble is used.
- **Hybrid mode**: local-first execution can reserve frontier APIs for disputed HIGH/CRITICAL findings.

The practical implication is not that automated patches should be merged without review. Rather, remediation suggestions can be generated entirely offline, and teams can reserve frontier review or manual audit time for the subset of findings and patches that survive local triage.

E. Towards Continuous Security

The illustrative pipeline's sub-10-second execution time suggests a practical CI/CD integration pattern. A GitHub Action running `miesc scan --ci` on every pull request, combined with `miesc fix --dry-run` to suggest patches in PR comments, would provide continuous security feedback without blocking development velocity. This integration pattern—detection as a gate, remediation as a suggestion—preserves developer autonomy while making security findings visible earlier in the development process.

F. Threats to Validity

We identify four threats to the validity of our evaluation:

Internal validity. (1) *LLM non-determinism*: the detection stage can use frontier LLMs, whose outputs may vary across

model versions and provider-side updates. The reproducible Paper 1 profile fixes the evaluated artifact and records layer selection, but future API runs may differ. (2) *Re-scan verification*: vulnerability elimination is measured by re-scanning the patched contract with the same pipeline family that detected the original vulnerability. A false negative on re-scan would inflate the elimination rate. The v2 baseline mitigates this by adding external Slither validation over every patched contract that compiles, reporting clean-HIGH results and residual HIGH findings separately from the MIESC re-scan rather than merging them into a single success metric.

External validity. (3) *Corpus selection bias*: SmartBugs-curated consists primarily of contracts from 2017–2019 with known vulnerabilities. These contracts are simpler (avg. 85 LoC) and more heavily studied than production DeFi protocols (avg. 500+ LoC with complex inheritance). Results on production code may differ, particularly for the compilation rate (which depends heavily on dependency availability). (4) *Vulnerability distribution*: the corpus over-represents reentrancy (31 contracts) and unchecked low-level calls (52 contracts) relative to their frequency in production code. Categories like oracle manipulation, MEV, and flash loan attacks—which dominate real-world exploits [17]—are absent from SmartBugs.

Construct validity. (5) *Compilation $\neq$ correctness*: a compiling patch is not necessarily semantically correct. The patch could introduce subtle behavioral changes (e.g., a reentrancy guard that blocks legitimate callback patterns). Our no-regression criterion measures only re-scan finding growth under the same pipeline, and the external Slither clean-HIGH metric measures only residual HIGH findings under one independent analyzer; neither proves preservation of all original functional behavior. (6) *Artifact granularity*: test generation, formal verification, and compliance reporting are evaluated end-to-end on an illustrative contract, while patch quality is measured corpus-wide. We therefore do not claim corpus-wide proof coverage for all generated artifacts.

Mitigation strategy. We address threats (3) and (4) by additionally reporting EVMBench results (120 business-logic vulnerabilities from real audit reports) in our companion paper [14], where MIESC achieves 92.5% recall on a corpus that includes oracle manipulation, flash loans, and governance attacks.

VI. Conclusion and Future Work

This paper reports how far the current MIESC remediation pipeline can move after detection, and where it still falls short. Evaluated on 143 contracts from SmartBugs-curated, the v2 baseline applies fixes to 123 contracts under the current scan (86%), compiles all 123 emitted patches standalone (100%), eliminates the target-category finding in 86/123 cases by MIESC re-scan (70%), satisfies the bounded no-regression criterion in 121/123 cases (98%), and reports 58/123 patched contracts clean of HIGH findings under external Slither validation (47%). The 47% external clean-HIGH result is intentionally reported as a separate metric: it shows that successful patch emission and compilation do not imply semantic closure under an independent analyzer. Compared to five prior repair tools (Table V), MIESC is the only system in our comparison that reports exploit tests, formal specifications, and compliance documentation alongside patches.

The design choice is to keep each stage inspectable. Detection informs patching, patching informs test generation, test generation checks the exercised exploit path, and formal verification can add stronger evidence when the generated specification is tractable. The current evidence supports this as a remediation-assistance workflow, not as a claim of fully automatic program repair.

Limitations. The v2 100% standalone compilation rate applies to emitted patches after current-scan filtering; it should not be read as proof that every historical SmartBugs contract requires or receives a patch. The text-level patching approach cannot restructure code (e.g., apply CEI pattern) or handle cross-contract vulnerabilities. SMTChecker verification is tractable on only a minority of contracts within the default timeout, and corpus-wide semantic equivalence remains future work.

Future work. Three directions are planned: (1) *AST-level patching* via Slither's intermediate representation, enabling CEI reordering and cross-contract fixes; (2) *dependency-aware compilation* with automatic import resolution from npm/forge registries, which would eliminate the dominant compilation failure mode; and (3) *CI/CD integration* as a GitHub Action providing continuous security feedback on pull requests. We also plan to extend the evaluation to production DeFi protocols with complex inheritance and cross-contract interactions.

MIESC is available under AGPL-3.0 at the project repository (link withheld for double-blind review).

Acknowledgment

Acknowledgments are omitted in this version for double-blind review. The authors thank the Ethereum security community and the developers of Foundry, Slither, and OpenZeppelin.

References

1) J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019, pp. 8–15.
2) G. Grieco, W. Song, A. Cygan, J. Feist, and A. Groce, "Echidna: Effective, usable, and fast fuzzing for smart contracts," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020, pp. 557–560.
3) B. Mueller, "Smashing Ethereum smart contracts for fun and real profit," in *HITB Security Conference*, 2018.
4) Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program

analysis," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. ACM, 2024, pp. 1–13.

5) X. Yu, H. Zhao, B. Hou, Z. Ying, and B. Shi, "Smart contract repair," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 29, no. 4, pp. 1–32, 2020.
6) T. D. Nguyen, L. H. Pham, J. Sun, Y. Lin, and Q. T. Minh, "sGuard: Towards fixing vulnerable smart contracts automatically," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 1349–1367.
7) M. Rodler, W. Li, G. O. Karame, and L. Davi, "Elysium: Context-aware bytecode-level patching to automatically heal vulnerable smart contracts," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2022, pp. 742–754.
8) L. Zhang, K. Li, K. Sun, D. Wu, Y. Liu, H. Tian, and Y. Liu, "ACFIX: Guiding LLMs with mined common RBAC practices for context-aware repair of access control vulnerabilities in smart contracts," *IEEE Transactions on Software Engineering*, vol. 50, no. 7, pp. 1837–1853, 2024.
9) S.-L. Li, Y.-T. Chen, and W.-C. Huang, "SCPatcher: Automated smart contract patching via RAG-enhanced LLM," *arXiv preprint arXiv:2604.00687*, 2025.
10) Paradigm, "Foundry: Ethereum development toolkit," https://github.com/foundry-rs/foundry, 2025.
11) V. Wüstholz and M. Christakis, "Harvey: A greybox fuzzer for smart contracts," https://arxiv.org/abs/2005.03350, 2020.
12) A. Gupta, V. Gottimukkala, D. Robinson, Paradigm, and OpenAI, "EVMBench: Evaluating AI agents on smart contract security," *arXiv preprint arXiv:2603.04915*, 2026. [Online]. Available: https://github.com/paradigmxyz/evmbench
13) M. di Angelo, T. Durieux, J. F. Ferreira, and G. Salzer, "SmartBugs 2.0: An execution framework for weakness detection in Ethereum smart contracts," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 2102–2105.
14) Anonymous Author(s), "MIESC: Multi-layer intelligent evaluation for smart contracts," *arXiv preprint*, 2026.
15) J. F. Ferreira, P. Cruz, T. Durieux, and R. Abreu, "SmartBugs: A framework to analyze Solidity smart contracts," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2020, pp. 1349–1352.
16) a16z crypto, "Halmos: Symbolic testing for EVM smart contracts," https://github.com/a16z/halmos, 2023.
17) N. Atzei, M. Bartoletti, and T. Cimoli, "A survey of attacks on Ethereum smart contracts (SoK)," *Proceedings of the 6th International Conference on Principles of Security and Trust (POST)*, pp. 164–186, 2017.